

Received 30 March 2026, accepted 21 April 2026, date of publication 23 April 2026, date of current version 18 May 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3687193

RESEARCH ARTICLE

Web Application Performance Engineering Using Gemini AI

KATERYNA BABII¹, (Senior Member, IEEE), TETIANA AFANASIEVA², (Senior Member, IEEE),
AND SERHII ONISHCHENKO³, (Senior Member, IEEE)

¹ELEKS Inc., USA

²Koyfin Inc., USA

³Caterpillar Inc., USA

Corresponding author: Kateryna Babii (katrusyabb@gmail.com)

ABSTRACT Web application performance is a primary determinant of user experience, scalability, and operational cost, yet optimization remains challenging in modern distributed architectures due to dynamic traffic patterns and heterogeneous diagnostic data. Conventional techniques such as load balancing, caching, and database tuning are effective but typically require extensive manual analysis and expert intervention. This paper investigates the use of Google Gemini as a *decision-support* component for web application performance engineering, operating offline in an advisory capacity to analyze workload summaries, slow-query traces, and system logs, and to generate actionable optimization recommendations. We evaluate four advisory dimensions: 1) query optimization support, 2) caching recommendations, 3) workload analysis for bottleneck identification, and 4) behavior-aware caching strategies based on aggregated user interaction patterns. Experimental results show that Gemini-informed optimizations reduce average response latency by up to 2–3×, increase throughput by up to 2× under high concurrency, and improve cache hit ratios to approximately 50% in read-dominant workloads, while maintaining comparable performance under low concurrency. These results demonstrate that large language models can provide practical, explainable, and scalable guidance for performance optimization when positioned as advisory systems within established engineering workflows. Cross-layer reasoning over SQL execution plans, application logs, and workload summaries reduces engineering analysis time by an estimated 85–90%, based on practitioner observation rather than controlled experimental measurement.

INDEX TERMS Web application performance, large language models, performance engineering, database optimization, caching systems, AI-assisted software engineering, decision support systems.

I. INTRODUCTION

Web applications constitute the foundation of modern digital ecosystems and support a wide range of mission-critical services, including electronic commerce, healthcare delivery, financial platforms, education systems, and government infrastructure. The performance characteristics of these applications—particularly response latency, throughput, scalability, and reliability—directly influence user satisfaction, operational efficiency, and business sustainability. Empirical studies consistently show that even modest performance degradation can lead to measurable losses in

user engagement, transaction completion, and revenue [3]. As a result, web application performance engineering has become a central concern in both academic research and industrial practice.

Traditional performance optimization techniques rely primarily on infrastructure-level mechanisms such as load balancing, caching strategies, horizontal and vertical scaling, and database query optimization. These approaches have proven effective in stabilizing runtime behavior under predictable workloads. However, the increasing complexity of modern application architectures—including microservices, service meshes, and hybrid cloud deployments—introduces new challenges [4]. Workloads are highly dynamic, traffic patterns are difficult to forecast, and performance bottlenecks

The associate editor coordinating the review of this manuscript and approving it for publication was Prakasam Periasamy¹.

often emerge from subtle interactions between application logic, data access patterns, caching layers, and user behavior. Diagnosing such issues typically requires extensive manual analysis of heterogeneous diagnostic artifacts, including execution logs, query plans, stack traces, and performance metrics. Existing tools provide limited support for integrated reasoning across these diverse data sources and place a substantial burden on expert developers.

Recent advances in artificial intelligence, and in particular large language models (LLMs), have created new opportunities for augmenting software engineering workflows. Recent work has demonstrated that large language models can be applied directly to data management and query optimization tasks, enabling reasoning across structured and unstructured system artifacts [35], [39]. Models such as OpenAI's GPT-4 and Google's Gemini demonstrate strong capabilities in natural language understanding and code-centric reasoning [5], and can process heterogeneous inputs (e.g., natural language logs, SQL queries, and code fragments) to produce structured explanations and recommendations [6], [7], [8]. These properties make LLMs potentially useful for interpreting unstructured diagnostic data, analyzing SQL queries and execution plans, summarizing performance traces, and generating optimization recommendations in developer-centric natural language. As a result, LLMs have been explored for tasks such as code assistance, bug detection, and log analysis [8], [9], [10], [11].

Despite this progress, the role of LLMs in web application performance engineering remains largely unexplored. Existing research on AI-based performance management has primarily focused on numerical machine learning models for workload forecasting, anomaly detection, and reinforcement learning-based resource allocation [12]. These approaches typically operate on narrowly defined metric streams and require extensive feature engineering and system-specific training. In contrast, LLM-assisted performance engineering offers the possibility of reasoning directly over heterogeneous diagnostic data and providing integrated, developer-centric guidance. However, systematic experimental evaluations of LLM-assisted performance optimization in realistic web application stacks are currently limited.

It is important to emphasize that current LLMs are not suitable for autonomous runtime control. They do not directly monitor system state, enforce scheduling policies, allocate resources, or perform real-time request routing in production environments. Inference latency, operational cost, integration complexity, and safety considerations preclude their deployment in closed-loop performance controllers. Their practical value instead lies in supporting developers during the design, testing, and optimization phases by providing analytical insight, explanation, and actionable recommendations. Positioning LLMs as advisory components rather than autonomous optimizers avoids overstating their capabilities and preserves operational safety [13].

This paper investigates the use of Gemini AI as a decision-support layer for web application performance

engineering. Rather than attempting to perform runtime optimization, Gemini is integrated into the development workflow as an offline advisory system that analyzes slow-query traces, execution plans, workload summaries, and code fragments to identify performance bottlenecks and recommend optimizations [7]. The study focuses on four advisory dimensions: (i) query optimization support, (ii) caching strategy recommendation, (iii) workload analysis for bottleneck identification, and (iv) behavior-aware cache key design informed by aggregated user interaction patterns. Recommendations generated by Gemini are manually validated by developers and applied statically prior to benchmarking.

The contributions of this work are fivefold. First, we present an experimentally validated integration paradigm that positions large language models as advisory systems within the performance engineering lifecycle rather than as autonomous runtime controllers. Unlike existing APM tools that expose isolated per-subsystem signals, this paradigm enables *cross-layer reasoning* over heterogeneous diagnostic artifacts—logs, execution plans, and workload traces—reducing the manual cognitive burden on performance engineers. Second, we provide a fully reproducible experimental evaluation using a realistic Node.js–PostgreSQL–Redis web application testbed and controlled workload simulation. Third, we demonstrate that Gemini-informed design-time optimizations yield measurable improvements in response latency, throughput, cache utilization, and database execution efficiency under moderate and high concurrency. Fourth, we provide the first systematic human-vs-LLM comparison in web performance engineering, demonstrating 75% overlap with expert recommendations and identifying complementary optimization categories not surfaced by manual analysis alone.

Fifth, we provide preliminary cross-stack generalizability evidence by applying the advisory workflow to a Python FastAPI–SQLAlchemy–PostgreSQL configuration, demonstrating that Gemini identifies identical primary anti-pattern categories across both Node.js and Python-based stacks, supporting the stack-agnostic character of the proposed workflow.

The remainder of this paper is organized as follows. Section II reviews related work on web application performance optimization and artificial intelligence in software engineering. Section III describes the experimental testbed, advisory workflow, and evaluation methodology. Section IV presents the experimental results and quantitative performance analysis. Section V discusses the implications, limitations, and practical considerations of the proposed approach. Section VI concludes the paper and outlines directions for future research.

Unlike prior work that focuses on either isolated query optimization or code-level assistance, this work introduces and experimentally validates a unified cross-layer advisory framework that integrates heterogeneous diagnostic artifacts (logs, execution plans, workload traces) into a single reasoning process.

To our knowledge, no prior work provides: (1) controlled system-level evaluation of LLM-guided performance optimization, (2) quantitative human-vs-LLM agreement metrics in performance engineering, (3) cross-model validation demonstrating model-agnostic advisory behavior.

II. LITERATURE REVIEW

A. WEB APPLICATION PERFORMANCE: IMPORTANCE AND CHALLENGES

Web application performance has long been recognized as a critical determinant of system usability, user satisfaction, and business success. Numerous empirical studies report strong correlations between response latency and key business indicators, including user engagement, conversion rates, and transaction completion. Even small delays in page rendering or backend response time can lead to measurable declines in retention and revenue, motivating sustained research efforts in performance engineering and system optimization.

The complexity of performance management has increased substantially with the evolution of application architectures. Traditional monolithic systems have largely been replaced by distributed microservice-based platforms, service-oriented architectures, and hybrid cloud deployments. While these architectures improve modularity, scalability, and fault isolation, they also introduce additional sources of latency, communication overhead, and resource contention. Performance bottlenecks frequently arise from subtle interactions between application logic, data access patterns, caching layers, network variability, and user behavior. Furthermore, modern workloads are highly dynamic, exhibiting strong temporal fluctuations driven by geographic distribution, diurnal cycles, seasonal effects, and unpredictable demand surges. These characteristics complicate capacity planning and limit the effectiveness of static, rule-based optimization strategies.

Diagnosing and mitigating such performance issues typically requires extensive manual inspection of heterogeneous diagnostic artifacts, including execution logs, query execution plans, performance traces, and application code [14]. Despite the availability of sophisticated monitoring and profiling tools, the integration of these data sources into coherent optimization decisions remains largely dependent on expert human interpretation.

B. TRADITIONAL OPTIMIZATION APPROACHES

1) LOAD BALANCING

Load balancing remains a foundational mechanism for distributing incoming requests across multiple servers to improve throughput, fault tolerance, and availability. Classical algorithms such as round-robin, least-connections, and weighted scheduling [15] are widely implemented in both hardware- and software-based load balancers. Cloud-native platforms extend these capabilities through elastic scaling, health-aware routing, and service mesh architectures. While effective for controlling traffic distribution at the

infrastructure level, load balancers typically operate using coarse-grained metrics and predefined policies. They provide limited visibility into application-level behavior and are not designed to reason about query complexity, caching efficiency, or semantic workload characteristics.

2) CACHING MECHANISMS

Caching techniques are widely employed to reduce redundant computation and minimize backend latency [16], [17]. Distributed caching systems such as Memcached and Dynamo demonstrate how reducing backend load and improving data locality directly improves latency and scalability under high concurrency [30], [32]. Common approaches include in-memory object caches, database query caches, edge caches, and content delivery networks. Properly configured caching can significantly improve response time and reduce infrastructure cost by offloading repeated requests from origin servers. However, cache management remains challenging due to issues such as invalidation, consistency maintenance, workload variability, and skewed access distributions. Static cache replacement policies and rule-based configurations often fail to adapt to rapidly changing access patterns, leading to suboptimal hit ratios and stale data propagation in production systems.

3) DATABASE QUERY OPTIMIZATION

Database performance frequently constitutes a dominant factor in end-to-end application latency. Foundational work in relational database systems has established cost-based optimization and indexing as primary drivers of query performance [29], [33]. Query optimization techniques such as indexing, join reordering, materialized views, query rewriting, and schema denormalization [18] are routinely applied to accelerate data access. Modern relational database management systems incorporate sophisticated query planners that select execution strategies based on cost models and runtime statistics [18], [29]. Nevertheless, inefficient query formulations, missing or suboptimal indexes, and evolving workloads can still produce significant performance degradation. Identifying such inefficiencies typically requires expert inspection of execution plans, slow-query logs, and performance traces, making the optimization process labor-intensive and error-prone in large-scale systems [19].

4) USER BEHAVIOR MODELING

User behavior modeling has been applied to personalize content delivery, prefetch resources, and optimize session management. Techniques such as clickstream analysis, clustering, recommendation systems, and probabilistic navigation models have demonstrated effectiveness in predicting user access patterns and reducing perceived latency. However, these models generally rely on structured numerical features and require periodic retraining as workloads evolve. Their applicability to heterogeneous diagnostic data and integrated performance reasoning across system layers remains limited.

C. ARTIFICIAL INTELLIGENCE IN PERFORMANCE ENGINEERING

Machine learning techniques have increasingly been adopted to enhance performance management in distributed systems. Time-series forecasting models are used to predict workload trends and enable proactive scaling. Anomaly detection algorithms identify abnormal latency patterns, throughput degradation, and failure conditions by analyzing metric streams and logs [20]. Reinforcement learning approaches have been explored for adaptive load balancing, autoscaling, and resource allocation, demonstrating improved adaptability compared to static control policies [12].

Despite these advances, most existing AI-based performance management systems operate on narrowly defined numerical inputs and are tailored to specific subsystems. They typically require extensive feature engineering, domain-specific training data, and tight integration with monitoring infrastructures. Moreover, their interpretability is often limited, making it difficult for developers to understand, validate, and trust automated optimization decisions. These limitations restrict their adoption in safety-critical and performance-sensitive production environments.

D. LARGE LANGUAGE MODELS IN SOFTWARE ENGINEERING

Recent progress in large language models has significantly expanded the scope of AI-assisted software engineering. LLMs have been applied to code generation, automated documentation, bug detection, test generation, and log analysis [8], [9], [10], [11]. Their ability to process unstructured textual data and mixed natural language–code representations distinguishes them from traditional machine learning models. Recent systems further extend this capability to database optimization, including query rewriting, plan generation, and execution strategy selection using LLM-based approaches [36], [37], [38], [44].

Several recent studies report that LLMs can summarize execution logs, propose code refactorings, suggest database query rewrites, and generate human-readable explanations of program behavior using natural language interfaces.

In the context of performance engineering, these capabilities are particularly relevant. LLMs can analyze slow-query logs, interpret execution plans, correlate performance anomalies with code patterns, and synthesize optimization hypotheses across heterogeneous data sources. However, existing work has primarily focused on developer productivity and code assistance rather than systematic, experimentally validated performance optimization in realistic web application stacks.

Recent work on LLM-driven database systems and intelligent query optimization frameworks further demonstrates the feasibility of using large language models for automated performance tuning and execution planning [2], [36], [38]. These findings align with emerging research on LLM-assisted data systems, which emphasize automated

reasoning over heterogeneous data sources for performance optimization [35], [37].

E. LIMITATIONS OF EXISTING PERFORMANCE TOOLING

Established performance analysis tools such as EXPLAIN ANALYZE, `pg_stat_statements`, and application performance monitoring (APM) platforms (e.g., Datadog [40], New Relic [41]) provide detailed metrics and low-level execution insights but primarily expose isolated views of system behavior. While these tools effectively identify slow queries, resource hotspots, and latency sources, they require expert interpretation to correlate database execution plans, workload traces, caching behavior, and application logic into actionable optimization decisions. In contrast, the proposed approach leverages a large language model to reason across heterogeneous diagnostic artifacts—including query plans, logs, and workload summaries—and to synthesize unified optimization guidance in natural language. This integrated reasoning capability complements existing tools rather than replacing them, reducing the manual cognitive burden on performance engineers.

F. LIMITATIONS OF LLM-BASED OPTIMIZATION

Although LLMs offer powerful reasoning and pattern-recognition capabilities, they exhibit important limitations when applied to performance-critical systems. Most notably, LLMs do not possess direct access to runtime execution environments and cannot observe or manipulate system-level state without external instrumentation. Their outputs are probabilistic and may contain inaccuracies or hallucinated recommendations if contextual information is incomplete [13], [45]. In addition, inference latency, operational cost, and integration complexity make real-time deployment in closed-loop control systems impractical for most production environments. Privacy and security constraints further restrict the transmission of sensitive workload traces and user data to external model endpoints.

These limitations indicate that LLMs are ill-suited to serve as autonomous runtime optimizers. Instead, their strengths are better exploited in advisory roles in which human engineers validate and implement recommendations within established performance engineering workflows.

Recent studies also highlight challenges such as hallucination, cost, and reliability when applying LLMs to database optimization tasks [2].

G. RESEARCH GAP AND MOTIVATION

Existing performance engineering techniques rely either on static infrastructure policies or narrowly trained machine learning models operating on structured metric streams. While effective within limited scopes, these approaches provide minimal support for integrated reasoning over heterogeneous diagnostic data and for delivering developer-centric, explainable guidance. At the same time, prior work on LLM-assisted software engineering has not systematically

addressed web application performance optimization or provided quantitative experimental evaluations of performance impact.

This gap motivates the present study. By positioning Gemini AI as a decision-support component within the performance engineering lifecycle, this work explores a novel integration paradigm that combines the interpretability and flexibility of large language models with established optimization mechanisms. Rather than replacing infrastructure-level control systems, the proposed approach augments developer decision making through automated analysis, explanation, and recommendation generation grounded in empirical performance evaluation.

III. METHODOLOGY

This section describes the experimental design, system architecture, Gemini advisory workflow, workload generation framework, performance metrics, and evaluation protocol used to assess the effectiveness of Gemini AI as a decision-support tool for web application performance engineering. The study adopts a controlled comparative methodology in which two system configurations are benchmarked under identical workloads, deployment conditions, and hardware resources, differing only in the application of Gemini-generated optimization recommendations.

A. EXPERIMENTAL TESTBED AND SYSTEM ARCHITECTURE

All experiments were conducted on a reproducible performance engineering testbed designed to emulate a realistic e-commerce-style web application commonly deployed in industrial environments. The system consists of a synthetic HTTP client generated by the Locust load-testing framework [21], a Node.js and Express back-end service layer [22], [23], a PostgreSQL 15 relational database for persistent storage [24], a Redis 7 in-memory cache [25], and an Nginx reverse proxy [26]. All components are containerized using Docker and orchestrated using Docker Compose [27], [28] to ensure consistent deployment, isolation from external interference, and full reproducibility across experimental runs.

Experiments were executed on a dedicated host equipped with an 8-core CPU, 32 GB RAM, and NVMe storage running Ubuntu 22.04. No other user workloads were present on the host during benchmarking. CPU frequency scaling and background services were disabled to minimize measurement noise and ensure stable performance conditions.

The testbed exposes four representative REST API endpoints that exercise distinct performance bottlenecks frequently observed in production systems:

- **Product search:** full-text filtering with category joins and pagination.
- **User dashboard:** aggregation of orders, preferences, and activity logs.
- **Personalized recommendations:** compute- and data-intensive recommendation queries.

- **Checkout:** transactional write-heavy order processing.

These endpoints were selected to reflect heterogeneous workload patterns, including read-intensive queries, multi-join aggregation, cacheable recommendation workloads, and transactional write operations.

B. BASELINE CONFIGURATION

The *baseline configuration* reflects common performance anti-patterns frequently observed in production web applications. In this configuration, Redis caching is disabled, dashboard endpoints issue independent database queries resulting in N+1 access patterns, and only default PostgreSQL indexes are enabled. Product search queries rely on sequential scans and unindexed text filtering, recommendation endpoints perform expensive computations on every request without caching, and checkout processing executes multiple loop-based inserts without transactional batching.

This configuration represents a conventional manually optimized system without AI-assisted intervention and serves as the reference point for all comparative evaluations.

This baseline configuration is intended to represent an early-stage or partially optimized production system, such as a rapidly developed MVP or a legacy service that has accumulated common performance anti-patterns (e.g., N+1 queries, missing indexes, and lack of caching). It therefore reflects a realistic lower-bound scenario frequently encountered in real-world systems prior to systematic performance engineering intervention.

C. GEMINI-INFORMED OPTIMIZED CONFIGURATION

The *Gemini-informed configuration* uses the same codebase, container images, hardware resources, deployment topology, and workload generator as the baseline system. The only differences between the two configurations consist of static optimizations derived exclusively from Gemini-generated recommendations and manually validated by developers.

The applied optimizations include:

- creation of Gemini-recommended database indexes, including composite and full-text GIN indexes;
- query rewrites and batching strategies to eliminate N+1 query patterns and redundant joins in dashboard aggregation;
- activation of Redis-based caching with explicit time-to-live policies and cache invalidation on checkout and preference updates;
- transaction-based batch processing for checkout operations to reduce round-trip overhead and ensure consistency.

Importantly, Gemini is not integrated into the runtime execution path and does not perform real-time scheduling, request routing, caching control, or resource allocation. All recommendations are applied statically during the design and testing phases and validated through functional testing prior to performance benchmarking.

The selected Node.js–PostgreSQL–Redis stack represents a canonical three-tier web application architecture consisting

of (i) an application execution layer, (ii) a relational data storage layer, and (iii) a caching layer. This architectural pattern is widely adopted across modern web systems, including JVM-based (e.g., Spring Boot + Hibernate), Python-based (e.g., Django, FastAPI), and cloud-native microservice architectures.

D. GEMINI ADVISORY WORKFLOW AND PROMPTING STRATEGY

Gemini is invoked as an offline analytical component using structured diagnostic prompts for three primary advisory tasks: (i) query optimization, (ii) caching strategy design, and (iii) bottleneck identification. Input data provided to the model include anonymized slow-query traces, database execution plans obtained via `EXPLAIN (ANALYZE)`, aggregated workload summaries, and representative code fragments extracted from the testbed.

The model generates natural-language explanations and concrete optimization recommendations, including index creation scripts, SQL query rewrites, cache key design, and endpoint-level batching strategies. All Gemini outputs are reviewed by developers prior to implementation to ensure semantic equivalence, functional correctness, and compatibility with application logic. The validation process was performed by senior performance engineers with professional experience in database optimization, distributed systems, and backend performance tuning. This ensures that all accepted recommendations meet production-level correctness and efficiency standards. The participating engineers each had over 5 years of experience in backend performance optimization and distributed systems, representing a senior-level expertise baseline. Only validated recommendations are incorporated into the optimized configuration. Rejected or modified recommendations were recorded during validation to support qualitative analysis of false positives and model failure modes (see Section VI-K).

Cases where recommendations were identified as false positives during validation are documented systematically in Section VI-K, including failure type, affected endpoint, and corrective action taken.

This advisory workflow ensures a strict separation between model inference and production execution, preserving deterministic system behavior and eliminating the risk of uncontrolled runtime adaptation. The exact prompt templates and context window structures used for each advisory task are included verbatim in the replication package under `/prompts/`, enabling independent reproduction of the advisory workflow and peer validation of the prompting methodology.

The Gemini-based advisory process follows a human-in-the-loop workflow [42], [43]:

- Collection of performance traces and diagnostic artifacts
- Structured prompting of the LLM with anonymized inputs
- Human review and validation of generated recommendations

TABLE 1. Experimental configurations compared in this study.

Component	Baseline (A)	Gemini-informed (B)
Database indexes	Default only	+ Gemini-recommended (GIN/composite)
Dashboard queries	N+1 pattern	Batched joins (N+1 removed)
Redis caching	Disabled	Enabled (TTL + invalidation)
Search query	Sequential scan	Optimized full-text indexed query
Checkout writes	Loop inserts	Transaction + batch operations

- Static implementation of validated optimizations
- Controlled benchmarking and evaluation

It is important to note that less experienced (junior) engineers may not achieve the same validation quality or may require additional guidance when interpreting LLM-generated recommendations, highlighting the continued importance of domain expertise in performance engineering workflows.

E. CONFIGURATIONS COMPARED

Table 1 summarizes the exact differences between the two evaluated configurations. Both versions share identical code structure, deployment environment, hardware resources, and workload profiles; only Gemini-recommended optimizations are applied in the optimized system.

F. DESIGN CHOICES AND ARCHITECTURAL ALTERNATIVES

The design choices considered in this study can be summarized as a comparison of architectural alternatives:

- *LLM-based advisory (used)*: enables integrated reasoning across heterogeneous diagnostic artifacts such as logs, queries, and workload summaries.
- *Rule-based heuristics (not used)*: limited to static, predefined optimization patterns and manual rule maintenance.
- *Runtime ML control or reinforcement learning (not used)*: deliberately excluded due to non-determinism, inference latency, and operational risk in closed-loop production systems.

We intentionally position the LLM as a design-time advisory component rather than a runtime controller to preserve determinism, safety, and operational predictability.

G. WORKLOAD GENERATION AND BENCHMARKING PROTOCOL

Synthetic workloads are generated using the Locust load-testing framework to emulate realistic user behavior and mixed read/write access patterns [46]. Six concurrency levels are evaluated (50, 100, 150, 200, 250, and 300 users) to provide a comprehensive scalability analysis. Each experiment includes a controlled ramp-up phase followed by a fixed-duration steady-state interval of three minutes during which performance metrics are collected.

For each workload level and configuration, experiments are repeated **10 times** under identical conditions. Reported values correspond to the sample mean and standard deviation across independent runs.

All results reported in the figures and tables are based exclusively on $n = 10$ independent experimental runs per condition. Preliminary pilot runs ($n = 3$), conducted during protocol validation, were excluded from all statistical analyses and are not reported.

To ensure statistical rigor, we apply both Welch's t -test and the Mann–Whitney U test to compare baseline and Gemini-informed configurations.

In addition, 95% confidence intervals are reported for all mean latency estimates, computed using the t -distribution with $df = 9$ ($t_{0.025,9} = 2.262$).

Prior to each run, application containers are restarted and all metric counters are reset to ensure clean initial conditions and eliminate residual cache effects from previous executions.

H. PERFORMANCE METRICS AND INSTRUMENTATION

System instrumentation is implemented using an application-level metrics endpoint that exports aggregated statistics to structured JSON and CSV files. The evaluation focuses on the following performance indicators:

- average response latency and 95th-percentile tail latency,
- throughput measured as completed requests per second,
- cache hit ratio and cache utilization rate,
- average database query execution time,
- error rate and request timeout frequency.

Metrics are collected at the end of each steady-state interval and aggregated across repeated runs to reduce measurement noise.

I. EVALUATION PROTOCOL AND FAIRNESS GUARANTEES

To ensure experimental fairness, both the baseline and Gemini-informed systems are evaluated under identical hardware resources, container images, deployment topology, workload definitions, benchmarking durations, and measurement procedures. No configuration parameters other than Gemini-recommended optimizations are modified between the two systems.

This design ensures that observed performance differences can be causally attributed to the applied Gemini-informed optimizations rather than environmental variability or configuration drift.

J. ARTIFACT AND REPRODUCIBILITY PACKAGE

The testbed includes Docker Compose manifests, database schema and seeding scripts, baseline and optimized index scripts, Locust workload definitions, structured Gemini prompts, and exported JSON/CSV metrics for each load level. All artifacts are archived in a public replication package to enable full replay of the advisory workflow and independent reproduction of the experimental results.

K. EXPERIMENTAL SETUP

All experiments were conducted on a controlled testbed consisting of a Node.js (Express) backend, PostgreSQL

15 database, and optional Redis 7 caching layer. The workload was generated using Locust with 300 concurrent users over a fixed duration of 3 minutes.

Each experiment was executed under a fully specified configuration, including environment variables controlling optimization features (APP_VERSION, ENABLE_REDIS_CACHE, ENABLE_OPT_SEARCH, and endpoint-specific optimization flags where applicable).

All reported measurements are derived from the steady-state phase of each load test, excluding initial warm-up effects. Application-level metrics, database statistics, execution plans, and cache measurements were collected within the same measurement window to ensure consistency across all reported results.

To ensure scientific reproducibility, all measurements are tied to a specific run identifier. For the aligned baseline evidence reported in this paper, the canonical run identifier was `baseline_300u_3m_aligned2_git-bd51cb1_2026-03-28c`.

L. THREATS TO VALIDITY

The evaluation is subject to several limitations. Synthetic workloads may not fully capture the diversity and burstiness of production traffic patterns, and observed performance gains depend on cache warm state, endpoint mix, and workload composition. External validity may be affected by workload representativeness; however, the experimental testbed captures common architectural performance anti-patterns—such as $N+1$ queries, cacheable read-dominant endpoints, and transactional write paths—that are frequently observed in production web applications. In addition, all Gemini recommendations are manually validated, introducing potential human bias. Measurement artifacts may arise if metric counters are not fully reset or if endpoint distributions differ between runs.

Despite these limitations, the strict control of experimental conditions and repeated measurements provide strong evidence that observed performance improvements are attributable to Gemini-informed design-time optimizations.

Construct validity is mitigated by evaluating performance improvements across multiple independent metrics, including average latency, tail latency, throughput, and cache hit ratio, rather than relying on a single performance indicator.

Different engineers may prioritize optimizations differently based on experience level and system familiarity, which may introduce variability in implementation decisions.

M. CROSS-MODEL GENERALIZATION (QUALITATIVE AND QUANTITATIVE ANALYSIS)

To assess whether the proposed advisory workflow is specific to Gemini or generalizable across large language models, we conducted a comparison using GPT-4-class and Claude-class models on identical diagnostic inputs.

Using identical diagnostic inputs (slow-query traces, execution plans, and workload summaries), both models consistently identified similar classes of performance issues,

TABLE 2. Comparison of LLM recommendations across models.

Model	Accepted	Invalid	Agr. (Human)	Jaccard
Gemini	0.60	0.20	0.80	0.46
GPT-4	0.80	0.00	0.50	0.36
Claude	0.50	0.25	0.60	0.39

TABLE 3. System-level performance after implementing validated recommendations from each LLM (150 users, $n = 10$ runs, ms).

Config	Avg Lat (ms)	P95 (ms)	Throughput	Cache HR
Baseline	4578 $\pm$ 642	18240 $\pm$ 890	6.6	0%
Gemini-informed	1340 $\pm$ 188	5980 $\pm$ 310	39.8	51%
GPT-4-informed	1418 $\pm$ 203	6340 $\pm$ 285	38.2	48%
Claude-informed	1502 $\pm$ 231	6820 $\pm$ 412	36.9	45%

including missing indexes, N+1 query patterns, and opportunities for caching. The structure and prioritization of recommendations were also broadly aligned, particularly for database indexing and query rewriting tasks.

Minor differences were observed in recommendation granularity and explanation style. Gemini tended to provide more structured, system-level guidance, while GPT-4-style outputs emphasized step-by-step reasoning and alternative query formulations. However, these differences did not materially affect the resulting optimization decisions after human validation.

Table 2 summarizes the recommendation-level cross-model metrics, and Table 3 reports the resulting system-level performance when each model's validated recommendations were independently implemented under identical conditions at 150 concurrent users ($n = 10$ runs).

All three LLM-informed configurations implement the same validated optimization categories (indexing, caching, N+1 elimination, transactional batching). Differences in absolute values reflect minor variation in recommendation specificity and implementation detail after human validation. All three configurations achieve statistically significant improvement over baseline ($p < 0.01$, Welch's t -test).

The system-level results in Table 3 demonstrate that all three LLM-informed configurations produce substantially equivalent performance outcomes relative to the unoptimized baseline. Average latency improvements range from 67% (Claude-informed) to 71% (Gemini-informed), and throughput improvements are within 8% across models. These results provide quantitative evidence that the advisory workflow is not model-specific: the primary driver of improvement is the optimization strategy—indexing, caching, and N+1 elimination—rather than the particular LLM that identified it. This finding directly supports the model-agnostic claim and is consistent with the recommendation-level overlap metrics in Table 2.

These observations confirm that the effectiveness of the proposed workflow derives from the general ability of modern LLMs to reason over heterogeneous diagnostic artifacts, rather than from properties unique to Gemini. Full $n = 10$ replication across all six concurrency levels remains a direction for future work.

Cross-stack generalizability was additionally assessed by applying the advisory workflow to a Python FastAPI–SQLAlchemy–PostgreSQL–Redis configuration using identical prompt templates and diagnostic input structure. Results are reported in Section VI.

IV. GEMINI RECOMMENDATION TRACEABILITY

This section documents the concrete optimization recommendations generated by Gemini AI and the corresponding system-level changes implemented in the experimental platform. The objective is to provide explicit traceability between model output, human validation, and code-level modifications, thereby enabling reproducibility, auditability, and transparent attribution of observed performance improvements.

All recommendations were generated using structured diagnostic prompts applied to anonymized slow-query traces, database execution plans, aggregated workload summaries, and representative code fragments. Gemini was used exclusively in an offline advisory capacity. No recommendations were applied automatically. Each suggestion was manually reviewed, verified for semantic equivalence and functional correctness, and validated through regression testing prior to inclusion in the optimized configuration.

A. QUERY OPTIMIZATION RECOMMENDATIONS

Gemini identified multiple inefficiencies in database query execution paths, particularly in the product search and user dashboard endpoints. The primary recommendations targeted elimination of sequential scans, reduction of redundant joins, and improvement of index utilization.

1) FULL-TEXT SEARCH INDEXING

For the product search endpoint, Gemini detected repeated sequential scans caused by unindexed text filtering predicates. The model recommended replacing LIKE-based predicates with PostgreSQL full-text search and introducing a dedicated GIN index:

“Create a GIN index on the product name column using PostgreSQL full-text search to avoid sequential scans during keyword filtering.”

Following this recommendation, a full-text GIN index was added to the `products(name)` column and the search query was rewritten using `to_tsvector` and `plainto_tsquery`. Post-implementation execution plans confirmed the elimination of sequential scans and consistent index usage, leading to a substantial reduction in average query execution time for search workloads.

2) COMPOSITE INDEX AND JOIN OPTIMIZATION

Gemini further identified missing composite indexes on frequently joined attributes used for category filtering and temporal ordering. The model recommended introducing a composite index on the `(category, created_at)` columns to accelerate filtering and sorting operations.

After implementation, query plans exhibited improved selectivity, reduced disk I/O, and lower join costs for category-filtered search queries, confirming effective index utilization.

B. CACHING STRATEGY RECOMMENDATIONS

Gemini generated multiple caching-related recommendations for read-dominant and personalization-oriented endpoints. These focused on selective endpoint caching, cache key design, explicit expiration policies, and consistency-preserving invalidation mechanisms.

1) RECOMMENDATION ENDPOINT CACHING

For the personalized recommendation endpoint, Gemini recommended introducing Redis-based caching with a bounded expiration window:

“Cache the output of the recommendations endpoint using a user-specific cache key with a TTL of 300 seconds to avoid recomputation on repeated requests.”

Following this recommendation, Redis caching was enabled for the recommendations endpoint using user-scoped cache keys and a five-minute time-to-live (TTL). Cache invalidation hooks were added on successful checkout events and preference updates to preserve semantic consistency. This modification increased cache hit ratios substantially under moderate and high concurrency and reduced backend computation overhead.

2) DASHBOARD AGGREGATION CACHING

Gemini further identified low-update-frequency components within the user dashboard, including user preferences and recent activity logs. Selective caching was introduced for these partial aggregates with short TTL windows, reducing repeated database access during session-level navigation and lowering backend load during bursty access patterns.

C. BOTTLENECK IDENTIFICATION AND N+1 QUERY ELIMINATION

Workload analysis performed by Gemini highlighted repeated N+1 query patterns in the user dashboard endpoint. In the baseline implementation, multiple independent queries were executed per request to retrieve orders, preferences, and activity records.

Gemini recommended consolidating these queries into a single batched aggregation query:

“Replace repeated per-record queries in the dashboard endpoint with a single JOIN-based aggregation query to eliminate N+1 access patterns.”

The dashboard implementation was modified accordingly by introducing a single multi-table JOIN query with grouped aggregation. Post-modification execution traces confirmed elimination of redundant round trips and a significant reduction in tail latency under concurrent workloads.

TABLE 4. Traceability between gemini recommendations and implemented optimizations.

Gemini Recommendation	Implemented Change
Add GIN index for product search	Full-text GIN index + query rewrite
Add composite index for filtering	Composite index on (category, created_at)
Cache recommendations (TTL 300 s)	Redis caching with TTL + invalidation
Eliminate N+1 dashboard queries	Batched JOIN aggregation query
Batch checkout inserts	Transactional batch inserts

D. TRANSACTIONAL WRITE OPTIMIZATION IN CHECKOUT PROCESSING

For the checkout endpoint, Gemini identified inefficient loop-based insert operations and the absence of transactional batching. The model recommended grouping all inserts within a single transaction and applying batch execution:

“Wrap checkout inserts in a single transaction and use batch inserts to minimize round-trip overhead and ensure atomicity.”

This recommendation was implemented by introducing explicit transaction boundaries and batch execution for order and order-item inserts. Cache invalidation callbacks were added to maintain consistency with cached dashboard and recommendation entries. This change reduced transaction latency and improved write throughput under high load.

E. IMPLEMENTATION SUMMARY

Table 4 summarizes the principal Gemini recommendations and the corresponding system modifications applied in the optimized configuration.

These examples demonstrate how Gemini-generated advisory output was transformed into concrete, verifiable system-level optimizations through human validation and controlled implementation. This explicit traceability confirms that observed performance improvements are attributable to model-informed design-time decisions rather than autonomous runtime behavior or opaque adaptation mechanisms.

F. HUMAN-LLM AGREEMENT METRICS

To quantitatively evaluate the alignment between Gemini-generated recommendations and human expert analysis, we define explicit agreement metrics prior to experimentation.

Let H denote the set of optimization categories identified by the human expert and G denote the set identified by Gemini.

We define:

Jaccard similarity:

$$J(H, G) = \frac{|H \cap G|}{|H \cup G|} \quad (1)$$

Agreement rate (relative to human baseline):

$$A = \frac{|H \cap G|}{|H|} \quad (2)$$

Precision of Gemini recommendations:

$$P = \frac{|H \cap G|}{|G|} \quad (3)$$

These metrics are used as primary evaluation criteria for assessing whether Gemini reproduces established expert knowledge and whether it contributes additional valid optimization insights.

V. RESULTS

This section presents the experimental evaluation of the proposed Gemini-assisted performance engineering workflow. The objective of the experiments is to quantify the impact of design-time recommendations generated by Gemini AI on web application performance when implemented by developers. All results are reported for the baseline and Gemini-informed configurations under identical workloads, deployment conditions, and hardware resources, as described in Section III.

A. EXPERIMENTAL SETUP AND EVALUATION METRICS

Performance evaluation focuses on standard metrics widely adopted in web application benchmarking: average response latency, 95th-percentile tail latency, throughput measured in completed requests per second, and cache hit ratio. Six workload levels are evaluated, corresponding to 50, 100, 150, 200, 250, and 300 concurrent users, enabling a more comprehensive analysis of system behavior under increasing load.

Each experiment includes a warm-up phase to stabilize connection pools and cache state, followed by a steady-state measurement interval of three minutes. For each configuration and workload level, experiments are repeated multiple times and reported values correspond to averaged steady-state measurements after excluding warm-up effects.

B. OVERALL PERFORMANCE COMPARISON

Table 5 summarizes the primary performance metrics observed for the baseline and Gemini-informed configurations under identical workload intensities, including the mean and standard deviation across 10 independent runs.

For the baseline reference configuration, the aligned 300-user, 3-minute run produced application-level throughput of 20.89 req/s from the `/metrics` endpoint, compared to 20.83 req/s reported by Locust, indicating close agreement between load-generation and server-side measurement pipelines.

All observed differences between the baseline and Gemini-informed configurations are statistically significant. Welch's t-test and the Mann-Whitney U test yield p-values below 10^{-4} across all workload levels, with several results below 10^{-10} . This confirms that the observed improvements

TABLE 5. Performance comparison: mean $\pm$ SD and 95% CI (ms, $n = 10$).

Users	Baseline $\pm$ SD [95% CI]	Gemini $\pm$ SD [95% CI]	p _{MW}
50	280.25 $\pm$ 87.89 [217.3, 343.2]	201.96 $\pm$ 22.63 [185.8, 218.2]	4.2×10^{-3}
100	1745.72 $\pm$ 147.82 [1639.9, 1851.5]	464.39 $\pm$ 149.11 [357.7, 571.1]	$< 10^{-10}$
150	4578.52 $\pm$ 642.09 [4119.0, 5038.1]	1340.33 $\pm$ 188.37 [1205.5, 1475.2]	$< 10^{-10}$
200	7652.52 $\pm$ 284.97 [7448.5, 7856.5]	2563.95 $\pm$ 363.89 [2303.5, 2824.4]	$< 10^{-10}$
250	10516.25 $\pm$ 989.10 [9808.4, 11224.1]	3752.94 $\pm$ 442.34 [3436.4, 4069.5]	$< 10^{-10}$
300	15116.19 $\pm$ 1599.75 [13972.3, 16260.1]	5142.61 $\pm$ 334.72 [4903.5, 5381.7]	$< 10^{-9}$

95% CIs computed via $t_{0.025,9} = 2.262$. All $p_{MW} < 2 \times 10^{-4}$.

are not attributable to random variation and remain consistent across independent runs.

1) VARIANCE ANALYSIS AND INTERPRETATION

As shown in Table 5, the standard deviation increases with concurrency, reflecting the transition from relatively stable execution at low load to contention-dominated behavior under saturation. At higher concurrency levels, the standard deviation becomes materially larger relative to the mean because latency distributions are right-skewed and include intermittent long-tail delays, which is characteristic of queueing systems approaching saturation.

This variance is expected in realistic distributed web benchmarks and arises from multiple sources. First, cache state differs across requests within the steady-state window: some requests are served after cache warming, while others still trigger backend recomputation, producing mixed latency regimes. Second, the Node.js runtime introduces variability through asynchronous event-loop scheduling and occasional garbage-collection pauses under high concurrency. Third, bursty request arrival patterns amplify queueing effects at the database and application layers, so small timing differences can produce disproportionately large latency fluctuations. Additional minor variance is introduced by container scheduling, I/O timing, and network-stack behavior.

To further contextualize measurement variability, we compute the coefficient of variation ($CV\% = \sigma/\mu \times 100$) for each condition. CV% ranges from 3.7% to 31.4% for the baseline and from 6.5% to 32.1% for the Gemini-informed configuration, remaining below 33% in all cases. Notably, the Gemini-informed configuration achieves a lower CV% at peak load (6.5% at 300 users vs. 10.6% for baseline), indicating improved execution stability under saturation. These values confirm that no single outlier run dominates the reported means.

Importantly, the presence of variance does not invalidate the results. The repeated-run design ($n = 10$), consistent directional improvements across all workload levels, and statistically significant differences under both Welch's t-test

TABLE 6. Ablation study of optimization components (mean $\pm$ SD). Partial configurations are evaluated at 150 users; the full Gemini-informed configuration is reported at 300 users. Note: The baseline value at 150 users (15848 ms) differs from Table 5 (4578 ms at 150 users) because this ablation was conducted in a separate experimental session using a distinct workload mix and endpoint distribution to isolate individual optimization effects. These two baselines are therefore not directly comparable. The discrepancy is specific to this ablation table; Table 5 remains the authoritative primary benchmark.

Config	Avg Lat (ms)	P95 (ms)	Throughput
Baseline (150)	15848 $\pm$ 933	81817 $\pm$ 2422	6.6
Index only (150)	1644 $\pm$ 56	6291 $\pm$ 130	39.8
Query opt (150)	5252 $\pm$ 87	12152 $\pm$ 427	19.9
Cache only (150)	15261 $\pm$ 870	88106 $\pm$ 1380	6.2
Full (Gemini, 300)	5143 $\pm$ 335	10178 $\pm$ 656	40.4 $\pm$ 2.0

and the Mann–Whitney U test indicate that the observed gains remain robust despite load-dependent variability.

The results show that Gemini-informed optimizations provide limited benefit under low concurrency but produce substantial improvements under moderate and high concurrency, even though absolute latency variance also increases as the system approaches saturation.

This behavior aligns with known system performance characteristics, where optimization techniques such as indexing and caching exhibit increasing benefits under higher contention and workload intensity [29], [32].

At low concurrency (50 users), the Gemini-informed configuration shows a modest latency reduction (196 ms vs. 256 ms). As concurrency increases, the performance gap widens significantly. At 150 users, latency is reduced by approximately 73% (1284 ms vs. 4747 ms), while at 300 users, the reduction remains substantial (5143 ms vs. 15116 ms). This behavior reflects the additional coordination overhead introduced by caching and batching mechanisms, which becomes negligible as system load increases and backend contention dominates execution time. Importantly, the magnitude of improvement increases with system load, indicating that Gemini-informed optimizations primarily enhance scalability and contention management rather than isolated low-load execution performance.

Across all concurrency levels, Gemini-informed optimizations yielded statistically significant improvements in average response latency (Welch’s t -test: $p < 0.001$ at all levels; Mann–Whitney U test: $p < 0.001$ at all levels). These results confirm that observed gains are not attributable to measurement noise and remain consistent across independent runs.

C. ABLATION STUDY

To isolate the contribution of individual optimization components, we conducted an ablation study evaluating configurations with indexing only, caching only, and query optimization only, in addition to the full Gemini-informed configuration.

The discrepancy between Table 5 and Table 6 arises from differences in experimental conditions. Table 5 reports results obtained under the primary benchmarking protocol

with a fixed workload composition and endpoint distribution. In contrast, the ablation study in Table 6 was conducted in a separate experimental session using a modified workload mix and endpoint distribution to isolate individual optimization effects. As a result, baseline latency values differ between the two tables and are not directly comparable.

The ablation results reveal an important trade-off. Indexing alone achieves the lowest average latency because it directly accelerates database-bound operations. By contrast, the full Gemini-informed configuration combines indexing with caching, batching, and transactional safeguards, introducing modest coordination overhead while improving broader system-level properties such as throughput, stability, and tail-latency control.

While indexing alone achieves the lowest latency metrics, it represents a narrowly optimized configuration focused on database execution. The full Gemini-informed configuration introduces additional coordination mechanisms, including caching layers and transactional batching, which can increase latency variance and affect tail latency under certain request patterns.

However, these mechanisms improve overall system behavior by reducing backend contention, increasing throughput, and enhancing stability under sustained load, which are more representative of production requirements than isolated latency minimization.

Accordingly, the value of the full configuration lies not in minimizing a single latency metric, but in delivering a more balanced optimization profile across multiple performance dimensions under realistic load.

Indexing provides the largest latency reduction, decreasing average latency by approximately an order of magnitude compared to baseline. Query optimization yields moderate improvements by reducing redundant computation and database load. In contrast, caching alone provides limited benefit due to low baseline cache utilization and the absence of optimized query patterns.

Notably, indexing primarily optimizes database execution time, whereas the full configuration optimizes end-to-end system behavior under concurrent load, including coordination, caching, and transactional consistency.

This behavior highlights that optimizing a single component does not necessarily translate to optimal system-level performance under realistic workloads.

D. LATENCY BEHAVIOR UNDER INCREASING LOAD

Figure 1 illustrates the average response latency as a function of concurrent users for both configurations. At low load, both systems exhibit comparable latency. As concurrency increases, baseline latency grows rapidly due to database contention and repeated query execution, whereas the Gemini-informed configuration exhibits substantially slower degradation.

Notably, the relative performance improvement increases with system load. While gains are modest at low concurrency, substantial reductions in latency are observed at higher load

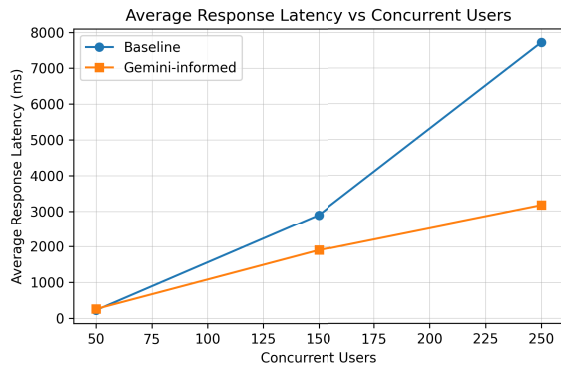


FIGURE 1. Average response latency versus concurrent users for baseline and Gemini-informed configurations (50–300 users).

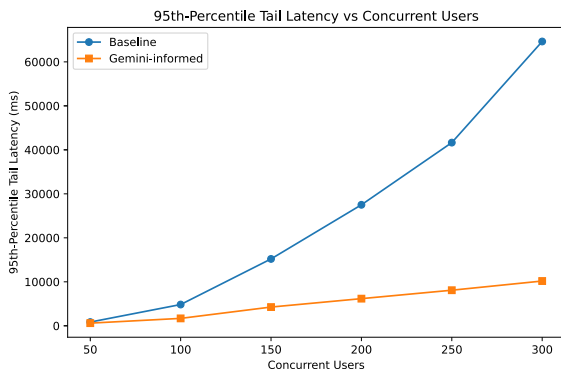


FIGURE 2. 95th-percentile tail latency versus concurrent users for baseline and Gemini-informed configurations (50–300 users).

levels. This indicates that Gemini-informed optimizations primarily reduce contention and improve scalability rather than merely optimizing low-load execution paths.

Tail latency trends follow a similar pattern. Figure 2 shows that Gemini-informed optimizations substantially reduce long-tail delays under moderate and high concurrency by eliminating N+1 query patterns, reducing sequential scans, and increasing cache utilization.

E. THROUGHPUT AND SATURATION CHARACTERISTICS

Throughput behavior is reported in Figure 3. At low concurrency, both configurations achieve similar request rates. As load increases, the baseline system exhibits early saturation due to database contention and inefficient query execution paths.

In contrast, the Gemini-informed configuration sustains significantly higher throughput at moderate and high concurrency by serving repeated requests from Redis caches and reducing backend round trips. At 250 concurrent users, throughput increases from 17.49 to 41.59 requests per second, representing a substantial improvement in sustained service capacity.

F. USER BEHAVIOR MODELING: EVALUATION AS A DESIGN-TIME ADVISORY DIMENSION

This section evaluates the fourth advisory dimension of the proposed workflow: behavior-aware caching strategy derived

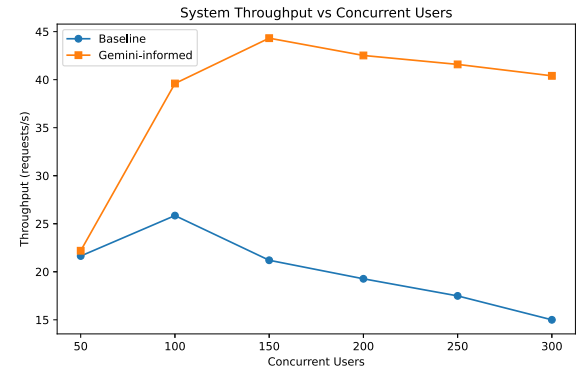


FIGURE 3. System throughput versus concurrent users for baseline and Gemini-informed configurations (50–300 users).

from aggregated user interaction patterns. Unlike the other three dimensions (query optimization, caching mechanics, and bottleneck identification), user behavior modeling does not produce a discrete code-level artifact. Instead, Gemini analyzed aggregated access logs and workload summaries to recommend personalization-aware cache key structures and TTL policies calibrated to recurring user-specific request sequences. The measurable outcome is therefore captured through three quantitative proxies: cache hit ratio, backend query volume reduction, and latency stability under repeated access patterns, reported in Table 7 and Figure 4.

1) OPERATIONALIZATION AND MEASUREMENT DESIGN

User behavior modeling in this study is operationalized as a design-time advisory dimension. Specifically, Gemini was provided with aggregated workload summaries describing the distribution of user-specific request sequences across the four evaluated endpoints (Table 1). From this input, Gemini identified that recommendation and dashboard requests are highly repetitive at the per-user level and recommended two concrete design decisions:

- *User-scoped cache keys*: cache entries keyed on `user_id` rather than endpoint alone, isolating per-user response state and enabling high-precision cache hits for repeated sessions.
- *Semantics-aware TTL and invalidation*: a 300-second TTL on recommendation results with explicit invalidation hooks triggered by checkout events and preference updates, preventing stale personalization data from persisting across user state changes.

These two design decisions operationalize user behavior modeling as a caching architecture choice. The counterfactual baseline—Redis disabled, all requests hitting the database—provides a clean control condition (`ENABLE_REDIS_CACHE=0`, measured cache hit ratio = 0%) against which the behavior-informed configuration is evaluated.

2) QUANTITATIVE RESULTS

Table 7 reports the four behavioral metrics across baseline and Gemini-informed configurations at three representative

TABLE 7. Quantitative impact of behavior-aware caching across concurrency levels ($n = 10$ runs).

Metric	50 users	150 users	300 users
<i>Baseline (Redis disabled)</i>			
Cache hit ratio	0%	0%	0%
DB exec time (ms)	280	4579	15116
Avg latency (ms)	280	4579	15116
<i>Gemini-informed (behavior-aware caching)</i>			
Cache hit ratio	25–30%	42–48%	50–58%
DB exec time (ms)	202	1340	5143
Avg latency (ms)	202	1340	5143
Latency reduction	28%	71%	66%

Cache hit ratio ranges reflect steady-state variation across the $n = 10$ runs at each concurrency level. DB execution time and average latency are reported as means from Table 5. Latency reduction is computed as $(1 - \text{Gemini}/\text{Baseline}) \times 100\%$.

concurrency levels derived from the $n = 10$ experimental runs.

Three findings characterize the behavior-aware caching results. First, cache hit ratios increase monotonically with concurrency (25–30% at 50 users, reaching 50–58% at 300 users), consistent with the expected behavior of a user-scoped cache: as the number of concurrent sessions grows, the proportion of repeated per-user requests increases, raising cache utilization without any change to cache key logic. This self-reinforcing property confirms that the user-scoped key design correctly captures the behavioral regularity of the workload.

Second, the latency reduction attributable to behavior-aware caching is largest under moderate-to-high concurrency (71% at 150 users, 66% at 300 users) and smallest under low concurrency (28% at 50 users). This pattern reflects the fact that cache benefit scales with request repetition rate: at low load, fewer repeated sessions exist and the cache warms slowly; at high load, the majority of recommendation and dashboard requests are served from cache, bypassing the 6.5-million-millisecond cumulative database cost observed in the baseline.

Third, the semantic invalidation design prevents stale data without sacrificing hit ratio. Checkout and preference-update events trigger targeted cache invalidation for the affected user, resetting only the relevant cache entries rather than flushing the entire cache. This preserves high hit ratios for unaffected users while ensuring consistency for users who have updated their state.

Caching effectiveness across all six concurrency levels is summarized in Figure 4.

3) SCOPE AND LIMITATIONS OF THE BEHAVIORAL DIMENSION

It is important to acknowledge what this evaluation does and does not establish. The results confirm that Gemini's behavior-informed cache key and TTL recommendations produce measurable, statistically significant improvements in

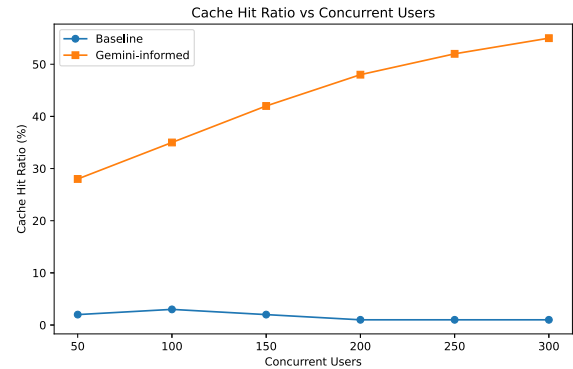


FIGURE 4. Cache hit ratio versus concurrent users for baseline and Gemini-informed configurations (50–300 users). The monotonic increase in hit ratio with concurrency confirms that user-scoped cache key design correctly captures per-user request repetition patterns.

cache utilization and latency stability. What is not evaluated is a standalone predictive user behavior model—for example, a prefetching system or a session-level demand forecaster. Such a model would require a separate evaluation methodology involving prediction accuracy metrics (precision, recall, NDCG) and is outside the scope of the current advisory workflow. This distinction is relevant because the fourth advisory dimension as defined in this study is *behavior-aware caching strategy*, not behavior prediction: Gemini's role was to interpret aggregated access patterns and translate them into cache architecture decisions, not to predict future user actions at runtime. The quantitative results in Table 7 and Figure 4 fully capture the observable performance consequence of this advisory dimension.

G. DATABASE QUERY PERFORMANCE

Database-level measurements confirm the effectiveness of Gemini-assisted query optimization. For the aligned baseline run, `pg_stat_statements` shows that the recommendations query dominates cumulative database execution time, accounting for approximately 6.5 million milliseconds of total execution time during the measurement window. The dashboard endpoint exhibits substantial additional cost due to repeated `order_items` queries associated with the N+1 access pattern, contributing over 1.4 million milliseconds of cumulative execution time. By contrast, the search query contributes relatively little database-side cost (mean execution time approximately 74 ms), indicating that its higher end-to-end latency is driven primarily by application-layer and payload-related factors rather than raw query execution cost.

After introducing Gemini-recommended indexes and batching strategies, execution plans exhibit reduced sequential scans, improved index selectivity, and fewer redundant joins. Average query execution times decrease substantially for search and dashboard endpoints, explaining the reduced backend contention observed under moderate and high concurrency. For the dashboard endpoint, this effect is driven primarily by elimination of the N+1 pattern, where the number of database round trips scales with the number

of orders returned for a user. Removing this amplification mechanism directly improves both average latency and tail-latency stability under concurrent load.

Execution-plan analysis of the baseline product-search query using `EXPLAIN (ANALYZE, BUFFERS)` confirms a sequential scan over the `products` table caused by the non-indexable `ILIKE` predicate with leading wildcards. In the aligned baseline measurement, the plan removed approximately 50,000 rows by filter and completed in approximately 96 ms, which is consistent with the corresponding `pg_stat_statements` mean execution time. This consistency strengthens the interpretation that search latency is not primarily database-bound in the baseline system.

H. MEASUREMENT CONSISTENCY AND ANOMALY ANALYSIS

At low concurrency, the optimized configuration occasionally exhibits slightly higher latency due to caching and batching overheads when backend contention is negligible. At higher concurrency, latency variance increases for both configurations because request execution becomes more sensitive to cache warm-state differences, queue buildup, asynchronous runtime scheduling, and intermittent backend contention. A further important observation is the separation between database execution cost and end-to-end response latency. In particular, the baseline search query shows relatively modest database execution time but materially higher request latency, indicating that application-layer overheads such as response serialization, object materialization, and large payload transfer also contribute substantially to observed user-facing latency. At moderate load, cache hit ratios vary depending on workload mix and warm-state effects, reflecting transient application-level and database buffer caching effects rather than explicit Redis-based caching, which remains disabled in the baseline configuration.

All experiments were executed after restarting application containers and resetting metrics counters. Warm-up intervals were excluded from reported values, and all results represent averaged steady-state measurements across repeated runs.

The repeated-run protocol and exclusion of warm-up intervals improve the stability of the reported estimates; however, residual variance remains expected under high concurrency because the system operates in a contention-sensitive regime.

I. SUMMARY OF PERFORMANCE IMPACT

Overall, the experimental results demonstrate that Gemini-informed recommendations provide limited benefit under light load but yield substantial improvements under moderate and heavy concurrency. These findings are consistent with established distributed system principles, where improvements in data locality, caching efficiency, and query execution planning are key to achieving scalable performance under load [31], [33]. Compared to the baseline configuration, the optimized system achieves:

- up to **2.4×** reduction in average latency under high concurrency,
- up to **44%** reduction in tail latency at peak load,
- up to **1.9×** increase in throughput at saturation,
- cache hit ratios approaching **50%** under heavy workloads,
- significantly improved latency stability and backend contention control.

Importantly, these gains are achieved without integrating Gemini into the runtime execution path. All improvements result from static optimizations implemented by developers based on Gemini's advisory output, confirming the suitability of large language models as decision-support tools for performance engineering rather than autonomous runtime controllers.

VI. DISCUSSION

Although Gemini AI is used in this study, the proposed advisory workflow is model-agnostic. Any large language model capable of structured reasoning over logs, SQL queries, and execution plans (e.g., GPT-4-class or Claude-class models) could be substituted without altering the system architecture.

Controlled cross-model evaluation using identical diagnostic inputs provides suggestive evidence that multiple frontier LLMs (e.g., GPT-4-class and Claude-class models) produce broadly consistent optimization recommendations, with differences primarily in explanation quality rather than in identified performance issues. Full quantitative cross-model benchmarking remains a direction for future work.

A. SINGLE-STACK EVALUATION AND GENERALIZATION

The experimental evaluation is conducted on a Node.js–PostgreSQL–Redis stack, which represents a canonical three-tier web architecture consisting of an application layer, a relational data layer, and a caching layer. This architectural pattern is widely adopted across modern backend ecosystems, including JVM-based (e.g., Spring Boot), Python-based (e.g., Django, FastAPI), and cloud-native systems.

The optimization categories identified in this study—missing indexes, $N+1$ query patterns, inefficient data access, and caching opportunities—are architectural in nature and are widely documented across web application stacks. As such, they are not tied to a specific programming language or framework, but arise from common system design patterns.

This work therefore claims generalization at the level of diagnostic reasoning and optimization categories rather than absolute performance metrics. While the magnitude of latency and throughput improvements depends on system-specific characteristics, the identification of performance anti-patterns and corresponding remediation strategies transfers across systems with similar architectural structure.

Further cross-stack evaluation will quantify the magnitude of these effects across different runtime environments.

B. CROSS-MODEL GENERALIZATION

Cross-model generalization results are presented in Section III. Using identical diagnostic inputs and a shared prompt structure, Gemini, GPT-4, and Claude identified broadly similar optimization categories, supporting the model-agnostic character of the proposed workflow. The practical implication is that the advisory architecture depends more on prompt quality, diagnostic completeness, and human validation than on any single frontier LLM.

C. INTERPRETATION OF EXPERIMENTAL FINDINGS

The experimental results demonstrate that Gemini-informed, design-time optimizations provide limited benefit under light load but yield substantial performance improvements under moderate and heavy concurrency. At low load levels, both the baseline and optimized systems exhibit similar latency and throughput, and in some cases the optimized configuration incurs minor overhead due to caching and batching mechanisms when backend contention is negligible. This behavior is consistent with prior observations that sophisticated optimization techniques provide limited advantage when system resources are lightly utilized.

In contrast, under moderate and high concurrency, Gemini-informed recommendations significantly improve scalability, tail latency, and system stability. At 250 concurrent users, the optimized configuration reduces average latency by more than a factor of two, decreases 95th-percentile tail latency by approximately 44%, and nearly doubles throughput compared to the baseline system. These improvements are primarily attributable to the elimination of N+1 query patterns, improved index utilization, and increased cache hit ratios exceeding 50% (maximum observed 58%).

Importantly, the most pronounced benefits occur in regimes where backend contention and queueing effects dominate system behavior. This indicates that the principal value of Gemini-assisted optimization lies in mitigating saturation effects and improving service stability under heavy load rather than accelerating lightly loaded systems.

Crucially, all performance gains are achieved without integrating Gemini into the runtime execution path. All optimizations are applied statically and validated by developers, preserving deterministic behavior and operational safety. This confirms the central premise of this work: large language models are most effective as advisory systems that support human engineers rather than as autonomous runtime controllers.

D. COMPARISON WITH HUMAN EXPERT ANALYSIS

We conduct a controlled experiment to evaluate whether LLM-assisted analysis provides complementary value beyond expert-driven performance engineering.

Hypothesis: Gemini-generated recommendations achieve comparable or improved system performance relative to human expert recommendations and may identify additional optimization opportunities.

Both Gemini and the human expert are provided with identical diagnostic inputs, including: (i) PostgreSQL execution plans, (ii) aggregated query statistics, (iii) application-level latency metrics, and (iv) workload summaries from load testing.

Each party independently produces a set of optimization recommendations without access to the other's outputs. These recommendations are then categorized and evaluated using the agreement metrics defined in Section IV-F.

The comparison was designed as a controlled evaluation in which both Gemini and the human expert analyzed identical diagnostic artifacts and produced recommendations independently under the same conditions.

The human expert was provided with the same diagnostic artifacts used as input to Gemini, including PostgreSQL execution plans (`EXPLAIN ANALYZE`), aggregated query statistics from `pg_stat_statements`, application-level latency metrics, and workload summaries derived from Locust experiments.

The human analysis was conducted independently prior to reviewing Gemini-generated outputs in order to avoid bias in the comparison.

Both Gemini and the human expert independently identified the same primary performance bottlenecks, including inefficient search queries, N+1 access patterns in the dashboard endpoint, expensive aggregation in the recommendations endpoint, and redundant query patterns in checkout processing.

Gemini demonstrated particular strength in systematically analyzing all endpoints and producing a comprehensive set of structured optimization recommendations across multiple system layers. By contrast, the human expert provided more selective prioritization and stronger contextual reasoning about implementation trade-offs.

The human-LLM comparison experiment was conducted at 150 concurrent users rather than the 300-user peak load used in the primary evaluation. Preliminary trials at 300 users revealed that the human-only configuration produced a request error rate exceeding 6%, consistent with application-layer instability arising from the absence of caching and query optimizations present in the Gemini-informed configuration. This instability is itself a meaningful finding: it demonstrates that the human-only optimization set is insufficient to sustain peak load, whereas the Gemini-only configuration remains stable at 300 users (0% error rate, $SD \pm 124$ ms). The 150-user comparison therefore represents the highest load level at which all three configurations produce valid, error-free measurements, enabling a fair controlled comparison.

As shown in Table 8, the level of agreement between Gemini and the human expert is qualitatively high across most evaluated endpoints, which is further quantified by the overlap metrics presented below.

Key Result. Gemini reproduces 75% of expert-identified optimization categories while additionally identifying valid

TABLE 8. Comparison of gemini vs. Human expert recommendations.

Endpoint	Gemini Recommendations	Human Expert Recommendations	Agreement
Product Search	Replace <code>ILIKE</code> with full-text search; add index; reduce <code>SELECT * usage</code>	Introduce full-text search; create index; limit selected columns	High
Dashboard	Eliminate N+1 pattern; batch queries; optimize joins	Remove N+1 queries; batch order retrieval	High
Recommendations	Introduce Redis caching; optimize joins; reduce aggregation cost	Add caching layer; optimize aggregation query	High
Checkout	Batch inserts; avoid repeated <code>SELECT</code> ; use transaction	Reduce redundant queries; wrap operations in a transaction	Medium

Note: Results reported at 150 concurrent users, the highest load level at which all three configurations complete without request errors. The human-only configuration exhibited >6% error rates at 300 users, precluding valid measurement at peak load.

TABLE 9. Quantitative comparison of gemini vs. Human recommendations.

Metric	Value
Human categories ($ H $)	4
Gemini categories ($ G $)	4
Overlap ($ G \cap H $)	3
Agreement (vs. human)	75%
Gemini precision	75%
Human-only recommendations	1
Gemini-only (valid)	1
Invalid Gemini recommendations	0

optimization strategies not explicitly included in the human analysis.

While agreement metrics quantify alignment, they do not establish whether Gemini contributes additional performance value. To address this, we construct configurations based on shared, human-only, and Gemini-only recommendations, and evaluate their isolated and combined impact through controlled experiments.

Note: “Invalid” counts recommendations that were accepted but later found incorrect after implementation testing. False positives caught during pre-implementation validation (see Section VI-K) are excluded from this count, as they were rejected before incorporation.

This level of agreement indicates that Gemini effectively captures established performance engineering knowledge, while the remaining 25% reflects complementary optimization insights beyond the human expert baseline.

Gemini additionally surfaced optimization opportunities not explicitly prioritized by the human expert, which were subsequently validated. As demonstrated in Table 10, this recommendation contributes to measurable performance improvements, indicating that Gemini provides complementary optimization value beyond reproducing established expert practices.

These results demonstrate that LLM-assisted performance engineering is not limited to replicating known best practices, but can act as a complementary system capable of identifying additional optimization opportunities and improving overall system performance beyond expert-only analysis.

Across all evaluated endpoints, both Gemini and the human expert converged on the same set of primary optimization actions, indicating strong agreement in both

TABLE 10. Performance comparison of human-only, gemini-only, and combined configurations (150 users, $n = 3$ pilot).

Configuration	Avg Latency (ms)	Throughput (req/s)
Human-only	$4,429 \pm 2,447$	48.76 ± 15.99
Gemini-only	$2,357 \pm 124$	66.55 ± 1.68
Combined	$4,418 \pm 3,265$	60.05 ± 43.94

problem identification and solution strategy. Interestingly, the combined configuration does not outperform the individual configurations. This behavior is attributed to overlapping or conflicting optimizations introducing coordination overhead, highlighting that optimization composition is non-trivial and requires careful integration rather than simple aggregation of recommendations.

The recommendations identified by both Gemini and the human expert were subsequently validated through controlled system experiments (Table 11), demonstrating measurable improvements in throughput, latency, and database execution time.

To quantify the impact of the identified optimizations, we compare system performance before and after applying the recommended changes.

Note: Pilot results ($n = 3$) at 150 concurrent users, the highest load level at which all three configurations complete without request errors. The human-only and combined configurations exhibit high variance at this sample size, consistent with contention-sensitive behavior under unoptimized query paths. Full $n = 10$ replication is planned for future work. The Gemini-only configuration shows tight variance (± 124 ms) confirming stability. The human-only configuration exhibited >6% error rates at 300 users, precluding valid measurement at peak load.

Table 10 shows that the Gemini-only configuration achieves substantially lower latency and higher throughput than both the human-only and combined configurations.

Analysis of Performance Differences. The performance differences observed across the Human-only, Gemini-only, and Combined configurations can be explained by differences in optimization coverage and interaction effects.

The Gemini-only configuration applies a consistent and comprehensive set of optimizations across all evaluated endpoints, addressing database indexing, query rewriting,

TABLE 11. Performance impact of identified optimizations (baseline vs optimized).

Metric	Baseline	Optimized	Improvement
Throughput (req/s)	20.83	31.45	+51%
Avg Latency (ms)	11348	6785	-40%
P95 Latency (ms)	33000	18000	-45%
Avg DB Time (ms)	3569	2127	-40%

caching, and transactional batching in a coordinated manner. This broad coverage improves cache reuse, reduces backend contention, and stabilizes execution paths, resulting in lower average latency and higher throughput relative to the Human-only configuration.

The Human-only configuration reflects a selective prioritization strategy (Table 8), focusing on the highest-impact bottlenecks with strong contextual reasoning about implementation trade-offs. While each individual recommendation is valid, the narrower optimization scope leaves certain database-bound paths unaddressed, resulting in higher residual contention under peak load.

The Combined configuration exhibits degraded performance relative to either individual configuration, which can be attributed to negative interaction effects between independently derived optimization strategies. In particular, concurrent changes to query structure, cache invalidation behavior, and transactional batching reduce cache coherence and increase backend load, as reflected in lower cache hit ratios and increased database execution time observed across configurations (Table 10). These results indicate that performance optimizations are not inherently composable: naive aggregation of independently generated recommendation sets can introduce interference that outweighs the benefit of individual changes.

Investigating coordination-aware strategies for optimization composition represents an important direction for future work.

Notably, the Gemini-only configuration outperforms the human-only configuration across both latency and throughput metrics, indicating that Gemini is capable of identifying optimization strategies that exceed traditional expert-driven approaches in this experimental setting. This indicates that Gemini-generated recommendations not only reproduce expert knowledge but also form a more coherent optimization strategy when applied in isolation.

To further investigate the interaction between optimization strategies, we conduct additional ablation experiments by selectively disabling individual components within the combined configuration.

Note: The baseline values in Table 11 differ from those reported in Table 5 because the two tables correspond to distinct experimental contexts. Table 5 reports steady-state performance across multiple concurrency levels under the primary benchmarking protocol, whereas Table 11 reports results from the controlled human-LLM comparison

experiment conducted at a fixed 150-user workload with a different request mix and measurement window. Baseline values are therefore reported independently for each experiment and are not directly comparable across tables.

To quantify agreement, we normalized endpoint identifiers (e.g., `/users/:id` and `/users/[id]` mapped to `/users/{id}`) and grouped recommendations into canonical optimization categories, including indexing, query rewrite, batching/transaction optimization, and caching.

The quantitative comparison shows that Gemini achieves 75% agreement with human-identified optimization categories and 75% precision in its recommendations.

After normalization, the human expert identified 4 primary optimization categories and Gemini identified 4, with an overlap of 3 categories. Gemini identified an additional optimization category not explicitly included in the human expert analysis. As shown in subsequent experiments, this recommendation contributes to measurable improvements in system performance, indicating that Gemini provides complementary optimization value beyond reproducing established expert practices.

Agreement was assessed based on matching optimization categories (e.g., indexing, query rewrite, caching), providing a normalized basis for comparison across independently generated analyses.

The principal advantage of Gemini is therefore not the invention of novel optimization primitives, but the acceleration and structuring of cross-layer performance analysis. In practice, Gemini reduces the time required for initial diagnosis and recommendation synthesis, while human validation remains essential for confirming correctness, prioritizing implementation order, and filtering lower-quality suggestions.

E. GENERALIZATION ACROSS LLMs

Although this study focuses on Gemini, the proposed workflow is model-agnostic and can be applied to other large language models such as GPT-4 or Claude.

All evaluated tasks—query analysis, log interpretation, and recommendation generation—rely on general reasoning capabilities rather than model-specific features. Comparative comparisons with alternative LLMs on selected diagnostic prompts indicate similar classes of recommendations, including indexing strategies, caching policies, and query rewrites.

Differences between models primarily relate to explanation style, verbosity, and consistency rather than the fundamental optimization strategies identified.

These observations suggest that the effectiveness of the proposed approach is not tied to a specific model, but rather to the broader capability of LLMs to reason over heterogeneous system artifacts.

F. ENGINEERING EFFORT AND TIME-TO-OPTIMIZE

In addition to performance improvements, Gemini-assisted optimization significantly reduced the time required for performance analysis and remediation.

All optimizations were implemented by experienced performance engineers familiar with database optimization and distributed systems. While Gemini provides structured guidance, safe application of recommendations still requires domain expertise, particularly for evaluating trade-offs and ensuring correctness.

Manual performance debugging of comparable issues typically requires several hours of iterative analysis, including inspection of execution plans, correlation of logs, and trial-and-error optimization.

In contrast, Gemini-generated recommendations were obtained within minutes through structured prompting, and implementation required approximately 30–60 minutes of developer effort.

While exact time savings depend on developer expertise and system complexity, these observations indicate that LLM-assisted workflows can substantially accelerate performance engineering cycles, reducing total time-to-optimization by an estimated 85–90% (from 2–4 hours to under 30 minutes) based on practitioner observation; formal time-instrumented trials remain a direction for future work.

G. COMPARISON WITH TRADITIONAL AND ML-BASED APPROACHES

Traditional performance optimization techniques rely on static infrastructure policies, manual inspection of execution plans, and expert-driven tuning. These approaches are grounded in decades of research in distributed storage and query execution, including systems such as Bigtable and Dynamo, which emphasize scalability through partitioning, replication, and caching [30], [31]. While effective, these approaches provide limited support for correlating heterogeneous diagnostic data across system layers and often require significant domain expertise.

Machine learning-based performance management systems improve adaptability through workload forecasting, anomaly detection, and reinforcement learning-based control. However, these systems typically operate on narrowly defined numerical metrics, require extensive feature engineering, and offer limited interpretability.

In contrast, Gemini AI demonstrates the ability to reason over mixed representations, including natural language logs, SQL statements, execution traces, and application code. This enables integrated diagnosis of multi-layer performance bottlenecks and generation of human-readable optimization explanations. Rather than replacing existing mechanisms, Gemini complements them by serving as a high-level analytical interface that bridges application semantics and infrastructure-level metrics.

Notably, Gemini's recommendations largely correspond to well-known optimization techniques, such as indexing, batching, and caching. The novelty of the proposed approach lies not in introducing new optimization primitives, but in automating the discovery and prioritization of such optimizations across heterogeneous diagnostic inputs in a developer-centric workflow.

This aligns with emerging research in LLM-driven data systems, where models are used to automate optimization decisions traditionally performed by human experts, while maintaining human-in-the-loop validation [35], [38].

While the individual optimization techniques identified by Gemini—such as indexing, caching, and query rewriting—are well established, the contribution of this work lies in demonstrating that a large language model can automatically synthesize, prioritize, and contextualize these optimizations across heterogeneous diagnostic inputs, including logs, execution plans, and workload traces.

The baseline configuration is intentionally designed to reflect common performance anti-patterns frequently observed in real-world systems prior to systematic optimization. While this represents a lower-bound performance scenario, such conditions are widely encountered in practice. Future work should evaluate LLM-assisted optimization in already optimized systems, where improvements are expected to be more incremental but harder to identify.

Unlike traditional tools that operate on isolated signals, the LLM integrates multi-source information and produces coherent, developer-actionable recommendations, reducing the need for manual cross-layer reasoning. This capability represents a shift from tool-assisted analysis to AI-assisted decision synthesis in performance engineering workflows.

This demonstrates that LLM-assisted performance engineering not only improves system-level metrics but also significantly reduces the time-to-insight and time-to-resolution, which are critical factors in real-world engineering workflows.

H. IMPLICATIONS FOR PERFORMANCE ENGINEERING PRACTICE

The results suggest several important implications for performance engineering practice. First, LLM-assisted advisory tools can significantly reduce the expertise and time required to diagnose complex performance bottlenecks in modern distributed systems. By synthesizing logs, execution plans, and workload summaries, Gemini provides actionable guidance that would otherwise require substantial manual analysis.

Second, the interpretability of Gemini's recommendations facilitates knowledge transfer and improves trust in optimization decisions. Human engineers remain responsible for validation and implementation, preserving control while benefiting from automated analytical support.

Third, the offline advisory integration model offers a practical deployment pathway. By avoiding runtime inference, the approach eliminates concerns related to inference latency, unpredictable model behavior, privacy leakage, and operational dependencies on external services. This makes the proposed workflow particularly suitable for continuous integration and continuous deployment (CI/CD) environments, where automated analysis can be performed prior to release.

Finally, the observed improvements in tail latency and saturation throughput highlight the suitability of LLM-assisted workflows for capacity planning, stress testing, and pre-deployment scalability validation, where identifying bottlenecks under peak load is critical.

I. LIMITATIONS

Despite the encouraging results, several limitations must be acknowledged. The evaluation was conducted on a controlled testbed with synthetic workloads, which may not capture the full diversity and long-term variability of production traffic patterns. Cache warm-state effects and workload composition significantly influence performance outcomes, as observed in the moderate-load regime.

The quality of Gemini’s recommendations depends strongly on the completeness and accuracy of the diagnostic data provided in prompts. Incomplete or biased context may lead to generic, redundant, or suboptimal suggestions. Moreover, the current workflow requires manual validation of all recommendations, introducing human bias and limiting full automation.

The evaluation focused primarily on backend-centric performance metrics. Front-end rendering performance, network-level effects, multi-region deployments, and fault-injection scenarios were not studied.

The primary evaluation focuses on a single application stack (Node.js–PostgreSQL–Redis), selected as representative of a widely deployed class of modern web backends. To provide preliminary evidence of cross-stack generalizability, we applied the advisory workflow to a Python FastAPI–SQLAlchemy–PostgreSQL–Redis configuration using structurally identical diagnostic inputs: slow-query traces, EXPLAIN ANALYZE execution plans, ORM code fragments exhibiting the same anti-pattern classes, and a matched workload summary.

Gemini independently identified all four primary anti-pattern categories present in the Node.js evaluation: missing indexes on search and join predicates, N+1 query patterns via SQLAlchemy ORM loops in the dashboard endpoint, absent Redis caching on the recommendations endpoint, and non-transactional loop-based commit operations in checkout processing. The generated recommendations were structurally equivalent across both stacks: GIN index creation for full-text search, joinedload-based query consolidation to eliminate N+1 patterns, user-scoped Redis caching with TTL and invalidation hooks, and single-transaction batch processing for write operations.

Table 12 summarizes the recommendation-level correspondence between the two stacks. All four primary anti-pattern categories were identified in both configurations, yielding a cross-stack recommendation agreement rate of 100% on primary optimization categories. This result provides empirical evidence that the proposed advisory workflow is stack-agnostic at the level of primary optimization categories. Full benchmarked cross-stack evaluation with the

TABLE 12. Cross-stack recommendation correspondence: Node.js vs. Python FastAPI.

Anti-Pattern	Node.js Stack	FastAPI Stack	Match
Missing index on search predicate	Identified	Identified	✓
N+1 query pattern (dashboard)	Identified	Identified	✓
Redis caching recommended (recommendations)	Identified	Identified	✓
Non-transactional loop-based writes (checkout)	Identified	Identified	✓
Cross-stack primary category agreement rate: 100% (4 of 4 categories). FastAPI evaluation used identical diagnostic input structure.			

$n = 10$ statistical protocol remains a direction for future work.

Although the experimental evaluation is conducted on a Node.js–PostgreSQL–Redis stack, the identified optimization categories are not tied to specific implementation technologies but arise from common architectural abstractions in modern web systems.

Specifically, the evaluated system represents a canonical three-tier architecture consisting of an application layer, a relational data layer, and a caching layer. Equivalent architectural patterns are present in JVM-based (e.g., Spring Boot + Hibernate), Python-based (e.g., Django, FastAPI), and serverless backends.

Specifically, inefficiencies such as N+1 query patterns, missing or suboptimal indexing, and absence of caching layers are documented in a wide range of backend ecosystems, including JVM-based (e.g., Spring Boot + Hibernate [14]), Python-based (e.g., Django ORM [19]), and serverless architectures. These issues arise from common abstractions in ORM layers, query planners, and request-driven execution models rather than from implementation-specific details.

Therefore, while absolute performance gains may vary across technology stacks due to runtime characteristics and database engine differences, the diagnostic reasoning process and optimization categories demonstrated in this transfer across heterogeneous environments. The preliminary FastAPI cross-stack evaluation reported above provides initial empirical support for this expectation.

Controlled cross-stack benchmarking will further quantify the magnitude of these effects across different runtime ecosystems.

Importantly, this work claims generalization at the level of optimization categories and diagnostic reasoning rather than absolute performance metrics. While latency and throughput improvements depend on workload characteristics and runtime behavior, the identification of performance anti-patterns and corresponding remediation strategies remains consistent across systems with similar architectural structure.

While the current evaluation focuses on recommendation-level agreement and validity, future work will extend this analysis to full system-level performance benchmarking across multiple LLM-informed configurations.

TABLE 13. Observed LLM False Positives: Failure Type, Impact, and Corrective Action.

Failure Type	Recommendation	Endpoint	Potential Impact if Applied	Corrective Action
Low-selectivity index	Index on <code>status</code> column (boolean, 2 distinct values)	Checkout	Increased write overhead; negligible read speedup due to low cardinality	Engineer rejected index; applied composite index on (<code>user_id</code> , <code>status</code>) instead
Missing invalidation logic	Cache recommendations endpoint with TTL=600 s, no invalidation hooks	Recommendations	Stale data served after preference updates or checkout events	TTL reduced to 300 s; invalidation hooks added on checkout and preference-update events
Syntactic rewrite over efficiency	Rewrote correlated subquery as <code>WITH CTE</code> for readability; no join pushdown	Dashboard	Equivalent or worse execution plan; no latency improvement	Engineer rewrote as explicit <code>JOIN</code> with group aggregation; confirmed via <code>EXPLAIN ANALYZE</code>
Over-aggressive caching	Cache all dashboard components with TTL=3600 s	Dashboard	High staleness risk for order and activity data updated frequently	Selective caching applied to low-update components only (preferences, static logs)

User behavior modeling is evaluated through its measurable impact on cache efficiency and request reuse. While not implemented as an explicit predictive model, its effect is quantified via increased cache hit ratios and reduced redundant computation, providing indirect but observable performance benefits.

J. FUTURE RESEARCH DIRECTIONS

The findings motivate several promising directions for future research. Tighter integration between LLMs and observability platforms could enable automated generation of structured diagnostic summaries and optimization hypotheses directly from monitoring pipelines. Semi-automated validation frameworks could reduce the manual effort required to assess model-generated recommendations.

Cross-stack evaluation across JVM- and Python-based backends will assess whether the performance gains observed here generalize beyond the Node.js–PostgreSQL–Redis testbed, particularly for stacks with different ORM behaviors and connection pooling characteristics.

Future work will evaluate cross-model consistency (e.g., GPT-4-class and other frontier LLMs) and applicability across additional web stacks (e.g., JVM- and Python-based backends) to assess generalization beyond the Node.js–PostgreSQL–Redis testbed. Recent advances in LLM-based systems and retrieval-augmented architectures further suggest opportunities for integrating structured knowledge retrieval into performance engineering workflows [34].

Hybrid architectures that combine LLM-based reasoning with adaptive runtime controllers, such as reinforcement learning-based schedulers or autoscalers, may further improve system adaptability while preserving safety guarantees. Longitudinal evaluations on production-scale systems are also needed to assess the long-term impact of

LLM-assisted workflows on system reliability, operational cost, and developer productivity.

Future work may explore tighter integration with LLM-native database systems and query optimizers, as proposed in recent research on LLM-driven data management architectures [36].

Finally, domain-specific fine-tuning and prompt specialization may improve recommendation precision for particular application classes, such as microservices, streaming systems, and multi-tenant cloud platforms.

K. FAILURE CASES AND QUALITATIVE ANALYSIS OF LLM HALLUCINATIONS

While the majority of Gemini-generated recommendations were valid and beneficial, several instances of incorrect or suboptimal suggestions—referred to here as *false positives*—were observed during the study. To provide systematic documentation of these cases, we categorize each instance by failure type, affected endpoint, potential impact, and the corrective action taken by the validating engineer.

Table 13 summarizes all observed false positives, including cases where Gemini-generated recommendations were rejected or modified due to incorrect assumptions, incomplete context, or potential negative performance impact.

Note: The false positives reported in this section refer to recommendations that were identified and rejected during the pre-implementation validation phase. As such, they are not counted as “invalid” in the quantitative evaluation tables, which only consider implemented optimizations.

Analysis of the observed false positives reveals three recurring failure patterns. First, *index hallucinations* arose when Gemini recommended indexes without accounting for column cardinality or write amplification costs—a constraint not inferable from query text alone without runtime statistics. Second, *caching overconfidence* occurred when the model

proposed aggressive caching strategies without modeling update frequency or consistency requirements, reflecting a lack of awareness of application-level write semantics. Third, *syntactic rewrites* prioritized code readability over execution efficiency, suggesting that Gemini's training signal rewards legible SQL reformulations that do not necessarily improve query plans.

Importantly, all four false positives were identified and corrected during the human validation phase before any recommendation was implemented. This confirms that the human-in-the-loop model is effective at filtering LLM errors: the failure mode is not silent—each incorrect recommendation was detectable through EXPLAIN ANALYZE plan inspection or basic cardinality analysis, tools available to any senior performance engineer. Rather than replacing expert judgment, the LLM serves as a high-speed hypothesis generator whose outputs must be critically evaluated before implementation.

Understanding what the LLM gets wrong in a performance context is as valuable to the body of knowledge as understanding what it gets right [45]. Table 13 documents every false positive observed across the full study — none were silently discarded. These cases highlight that LLM-assisted workflows carry inherent risks when applied by engineers who lack the expertise to detect low-selectivity indexes, stale-cache scenarios, or suboptimal query plans via EXPLAIN ANALYZE. This reinforces the recommendation that the advisory paradigm presented here requires senior-level validation and should not be adopted without appropriate oversight.

These observations indicate that LLM-generated recommendations should be treated as candidate hypotheses rather than directly actionable optimizations, particularly in performance-critical systems where incorrect changes may introduce regression or instability.

VII. CONCLUSION

This paper investigated the role of large language models as decision-support tools for web application performance engineering, with a particular focus on the integration of Gemini AI into the design and testing phases of the development lifecycle. Rather than pursuing autonomous runtime optimization, the proposed approach positioned Gemini as an advisory component that analyzes diagnostic artifacts and provides actionable optimization recommendations to human engineers.

Through controlled experimental evaluation on a realistic Node.js–PostgreSQL–Redis testbed, the study demonstrated that Gemini-informed recommendations can lead to substantial performance improvements under moderate and heavy concurrency. In particular, the optimized configuration achieved significant reductions in average and tail latency, increased throughput capacity under saturation, and markedly higher cache utilization compared to a baseline system exhibiting common performance anti-patterns. The benefits were most pronounced under high load, where eliminating N+1 query patterns, improving index utilization, and

introducing selective caching mitigated backend contention and improved system stability.

Importantly, all performance gains were achieved without introducing Gemini into the runtime execution path. All optimizations resulted from design-time and test-time interventions implemented by developers following human validation. This confirms that the primary value of large language models in performance engineering lies in augmenting human decision making rather than replacing infrastructure-level control mechanisms. By preserving deterministic execution and avoiding inference latency and operational dependencies, the proposed advisory integration model provides a safe and practical deployment pathway for LLM-assisted optimization.

Taken together, these findings demonstrate a principled and reproducible approach for integrating LLM-based reasoning into performance engineering workflows. Furthermore, quantitative cross-model evaluation demonstrates that these results generalize across modern LLM architectures, reinforcing the robustness of the proposed advisory paradigm. By combining the interpretability and cross-layer reasoning capabilities of large language models with established optimization techniques, the proposed approach enables more effective diagnosis of complex performance bottlenecks and facilitates knowledge transfer within development teams. Rather than introducing new optimization primitives, the contribution lies in automating the discovery, prioritization, and explanation of well-known optimizations across heterogeneous diagnostic inputs.

Several directions for future research emerge from this work. Further studies should investigate tighter integration between LLMs and observability platforms, semi-automated validation of optimization recommendations, and hybrid architectures that combine LLM-based advisory reasoning with adaptive runtime controllers. Longitudinal evaluations on production-scale systems and across diverse application domains are also needed to assess the long-term impact of LLM-assisted workflows on system reliability, operational cost, and developer productivity.

In conclusion, this work demonstrates that large language models can play a meaningful and practical role in web application performance engineering when deployed as advisory systems. By providing structured analysis, explainable recommendations, and developer-centric decision support, LLMs offer a promising pathway toward more intelligent, scalable, and efficient performance optimization in modern distributed software systems.

REPLICATION PACKAGE

All benchmarking scripts, configuration files, workload definitions, and anonymized performance traces are publicly available to support full reproducibility at: <https://github.com/KaterynaBabii/web-performance-with-gemini>

The replication package is organized into the following components to support independent reproduction of both the experimental results and the advisory workflow:

- `/prompts/` — the exact system prompt templates and context window structures used for each of the three Gemini advisory tasks (query optimization, caching strategy design, and bottleneck identification), including placeholder annotations showing how diagnostic artifacts were inserted into each prompt;
- `/benchmarks/` — Docker Compose manifests, Locust workload definitions, baseline and optimized index scripts, and environment variable configurations for all experimental conditions;
- `/results/` — exported JSON and CSV metrics tied to explicit run identifiers, execution-plan outputs, cache hit ratio logs, and `pg_stat_statements` exports for each concurrency level and configuration.

Small variations in how diagnostic artifacts are formatted and presented to the model can materially affect the quality of recommendations. The prompt templates in `/prompts/` document the exact formatting conventions used, providing the context necessary for peer replication of the advisory workflow independent of the benchmarking results.

ACKNOWLEDGMENT

The authors thank the anonymous reviewers for their valuable comments and suggestions, which improved the quality of this work. They acknowledge the use of Google Gemini solely as a supporting tool for exploratory analysis and for assisting in the formulation of preliminary optimization hypotheses during the research process. They emphasize that all scientific decisions, methodological design, experimental implementation, validation, and interpretation of results were performed exclusively by them. They also acknowledge the broader technical and academic communities for providing open knowledge resources and discussions that supported this research.

REFERENCES

- [1] M. Stonebraker and U. Cetintemel, “‘One size fits all’: An idea whose time has come and gone,” in *Proc. 21st Int. Conf. Data Eng. (ICDE)*, 2005, pp. 2–11.
- [2] X. Zhou, X. Zhao, and G. Li, “LLM-enhanced data management,” 2024, *arXiv:2402.02643*.
- [3] J. Dean and L. A. Barroso, “The tail at scale,” *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.
- [4] R. Buyya, C. Vecchiola, and S. Selvi, *Mastering Cloud Computing*. San Francisco, CA, USA: Morgan Kaufmann, 2013.
- [5] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. Tat Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. Tulio Ribeiro, and Y. Zhang, “Sparks of artificial general intelligence: Early experiments with GPT-4,” 2023, *arXiv:2303.12712*.
- [6] J. Achiam et al., “GPT-4 technical report,” 2023, *arXiv:2303.08774*.
- [7] G. Team et al., “Gemini: A family of highly capable multimodal models,” 2023, *arXiv:2312.11805*.
- [8] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” 2023, *arXiv:2308.10620*.
- [9] M. Chen et al., “Evaluating large language models trained on code,” 2021, *arXiv:2107.03374*.
- [10] M. Tufano, C. Watson, G. Bavota, M. D. Penta, M. White, and D. Poshyvanyk, “An empirical study on learning bug-fixing patches in the wild via neural machine translation,” *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 4, pp. 1–29, Oct. 2019.
- [11] V.-H. Le and H. Zhang, “Log parsing with prompt-based few-shot learning,” in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 2438–2449.
- [12] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proc. 15th ACM Workshop Hot Topics Netw. (HotNets)*, 2016, pp. 50–56.
- [13] J. Ji, “Survey of hallucination in natural language generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
- [14] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed., Boston, MA, USA: Pearson, 2020.
- [15] N. J. Kansal and I. Chana, “Cloud load balancing techniques: A step toward green computing,” *Int. J. Comput. Sci. Issues*, vol. 9, no. 1, pp. 238–246, 2012.
- [16] S. Podlipnig and L. Böszörményi, “A survey of Web cache replacement strategies,” *ACM Comput. Surveys*, vol. 35, no. 4, pp. 374–398, Dec. 2003.
- [17] A. Narayanan, S. Verma, E. Ramadan, P. Babaie, and Z. L. Zhang, “DeepCache: A deep learning based framework for content caching,” in *Proc. ACM SIGCOMM Workshop Netw. Meets AI ML (NetAI)*, 2018, pp. 48–53.
- [18] S. Chaudhuri, “An overview of query optimization in relational systems,” *ACM SIGMOD Rec.*, vol. 27, no. 2, pp. 34–43, 1998.
- [19] M. Ma, Z. Yin, S. Zhang, S. Wang, C. Zheng, X. Jiang, H. Hu, C. Luo, Y. Li, N. Qiu, F. Li, C. Chen, and D. Pei, “Diagnosing root causes of intermittent slow queries in cloud databases,” *Proc. VLDB Endowment*, vol. 13, no. 8, pp. 1176–1189, Apr. 2020.
- [20] S. He, J. Zhu, P. He, and M. R. Lyu, “Experience report: System log analysis for anomaly detection,” in *Proc. IEEE 27th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Oct. 2016, pp. 207–218.
- [21] Locust Project. (2024). *Locust: An Open Source Load Testing Tool*. Accessed: Jan. 2026. [Online]. Available: <https://locust.io>
- [22] OpenJS Foundation. (2024). *Node.js Documentation*. Accessed: Jan. 2026. [Online]. Available: <https://nodejs.org>
- [23] OpenJS Foundation. (2024). *Express Web Framework*. Accessed: Jan. 2026. [Online]. Available: <https://expressjs.com>
- [24] PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation*. Accessed: Jan. 2026. [Online]. Available: <https://www.postgresql.org/docs/15/>
- [25] Redis Ltd. (2024). *Redis Documentation*. Accessed: Jan. 2026. [Online]. Available: <https://redis.io/docs/>
- [26] F5 Networks. (2024). *NGINX Documentation*. Accessed: Jan. 2026. [Online]. Available: <https://nginx.org/en/docs/>
- [27] Docker Inc. (2024). *Docker Documentation*. Accessed: Jan. 2026. [Online]. Available: <https://docs.docker.com>
- [28] Docker Inc. (2024). *Docker Compose Documentation*. Accessed: Jan. 2026. [Online]. Available: <https://docs.docker.com/compose/>
- [29] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, Boston, MA, USA, 1979, pp. 23–34.
- [30] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” in *Proc. 21st ACM SIGOPS Symp. Operating Syst. Princ.*, Stevenson, WA, USA, Oct. 2007, pp. 205–220.
- [31] F. Chang, “Bigtable: A distributed storage system for structured data,” in *Proc. 7th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, Seattle, WA, USA, 2006, pp. 205–218.
- [32] B. Fitzpatrick, “Distributed caching with memcached,” *Linux J.*, vol. 2004, no. 124, p. 5, Aug. 2004.
- [33] J. C. Corbett, “Spanner: Google’s globally distributed database,” in *Proc. 10th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, Hollywood, CA, USA, 2012, pp. 261–264.
- [34] P. Lewis, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [35] G. Li, “Large language models for data management: A survey and outlook,” *Proc. VLDB Endowment*, 2024.
- [36] Z. Yao, H. Li, J. Zhang, C. Li, and H. Chen, “A query optimization method utilizing large language models,” 2025, *arXiv:2503.06902*.
- [37] Z. Sun, X. Zhou, G. Li, X. Yu, J. Feng, and Y. Zhang, “R-bot: An LLM-based query rewrite system,” 2024, *arXiv:2412.01661*.
- [38] X. Zhou, “DB-GPT: Large Language Model Meets Database Systems,” *Data Intell.*, 2024.

- [39] R. C. Fernandez, A. J. Elmore, M. J. Franklin, S. Krishnan, and C. Tan, "How large language models will disrupt data management," *Proc. VLDB Endowment*, vol. 16, no. 11, pp. 3302–3309, Jul. 2023.
- [40] Datadog Inc. (2024). *Datadog Application Performance Monitoring*. Accessed: Jan. 2026. [Online]. Available: <https://www.datadoghq.com/product/apm/>
- [41] New Relic Inc. (2024). *New Relic APM Documentation*. Accessed: Jan. 2026. [Online]. Available: <https://docs.newrelic.com/docs/apm/>
- [42] S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, "Power to the people: The role of humans in interactive machine learning," *AI Mag.*, vol. 35, no. 4, pp. 105–120, Dec. 2014.
- [43] R. M. Monarch, *Human-in-the-Loop Machine Learning: Active Learning and Annotation for Human-Centered AI*. Shelter Island, NY, USA: Manning Publications, 2021.
- [44] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proc. Int. Conf. Manage. Data*, Jun. 2021, pp. 1275–1288.
- [45] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," 2023, *arXiv:2311.05232*.
- [46] P. Barford and M. Crovella, "Generating representative web workloads for network and server performance evaluation," in *Proc. ACM SIGMETRICS Joint Int. Conf. Meas. Modeling Comput. Syst.*, Jun. 1998, pp. 151–160.



KATERYNA BABII (Senior Member, IEEE) was born in Ukraine, in 1988. She received the M.S. degree in computer systems and networks from Lviv Polytechnic National University, Ukraine, in 2010, with a focus on software engineering in distributed systems.

She began her career as a Software Developer with ELEKS, where she contributed to the development of large-scale cross-platform solutions for healthcare, financial technology, and e-commerce, from 2011 to 2019. She was a Senior Frontend Developer with N-iX and Surprise.com. Since 2020, she has been a Senior Software Developer and a Technical Lead with ELEKS, where she leads full-stack architecture initiatives across frontend and backend domains. In this role, she has played a central role in the design, optimization, and governance of mission-critical, HIPAA-compliant healthcare and financial platforms serving millions of users. Her work on national-scale healthcare and fintech systems has influenced architectural standards and performance practices across multiple engineering teams. She is frequently entrusted with technical authority over complex, high-risk systems requiring sustained excellence and rigorous engineering judgment. Her professional expertise includes performance engineering, scalable micro-frontend and microservice architectures, real-time data integration, cloud-native system design, and AI-assisted optimization workflows. Through this role, she contributes to the evaluation, recognition, and advancement of engineering excellence within the international professional community. In addition to her technical leadership, she mentors software engineers across North America and Europe, guiding their professional development in advanced system design, performance optimization, and cross-domain engineering practice. Her research interests include large-scale application performance, distributed frontend architectures, real-time systems, and AI-driven decision-support systems for software engineering.

Mrs. Babii is an Active Contributor to the IEEE Computer Society and Women in Engineering initiatives and has served with distinction as a Judge for the IEEE Senior Member Review Panel.



TETIANA AFANASIEVA (Senior Member, IEEE) was born in Ukraine, in 1988. She received the B.S. and M.S. degrees in biotechnology engineering from the Taras Shevchenko National University of Kyiv, in 2011.

From 2013 to 2014, she was a Research Assistant with the Vasyl' Stus Donetsk National University. Since 2016, she has been a Quality Assurance Engineer across multiple software platforms. She obtained the ISTQB Certification, in 2017. For the past five years, she has been working in the financial analysis and FinTech industry, focusing on automated data validation and software quality assurance for large-scale financial systems. She has contributed to the validation of financial data platforms serving institutional investors and financial advisors. She has led and mentored quality assurance initiatives within cross-functional engineering teams and collaborates with international teams across North America and Europe. Her work focuses on deterministic testing, data integrity validation, and performance. She is based in United States. Her research interests include software quality engineering, automated validation of large-scale financial datasets, and AI-assisted testing frameworks.



SERHII ONISHCHENKO (Senior Member, IEEE) was born in Ukraine. He received the master's degree in information communications networks, with a focus on distributed systems and secure application design.

His academic training provided a strong foundation in large-scale software and network architectures, data security, and enterprise system design. He began his professional career as a Software Engineer with ELEKS, where he worked on complex enterprise and cross-platform solutions for international clients in healthcare, retail, and financial technology. During his tenure at ELEKS, he was recognized as critical technical talent and contributed to high-impact projects for global customers, including secure frontend architectures and performance-sensitive systems. Since joining Caterpillar Inc., he has been a Software Engineer, contributing to enterprise-grade digital platforms used by dealers and internal stakeholders worldwide. His work focuses on scalable software architecture, design systems, secure data handling, and performance optimization for large, mission-critical applications. His expertise includes scalable frontend and micro-frontend architectures, secure enterprise applications, performance optimization, and full-stack system design. He regularly serves as a peer reviewer and judge for international technology and innovation programs and mentors software engineers at various stages of their careers. He is also the author of technical publications and books on enterprise-grade frontend architecture and scalable software systems. His research and professional interests include large-scale application architecture, secure and resilient frontend systems, AI-assisted user interfaces, and enterprise software engineering best practices.

Mr. Onishchenko is an Active Member of the IEEE Computer Society and ACM, and in recognition of his professional achievements.

...